Lab 08 - Object Detection and Quantitative Evaluation

Course: Generative AI for Image and Video Creation (TGS-2020505925) | Version v11.0 | 6 September 2026 Topic 04: Generative AI Video Creation, Animation and Editing Outcome: ELO4 | In-class time: 60 minutes of scheduled practical time

Competency	Statement
Ability A5	Design and apply machine-learning based methods for object detection, object tracking and activity recognition
Knowledge K8	Deep learning concepts
Knowledge K9	Object segmentation, detection and recognition

Scenario

Two machine-learning detectors that ship inside OpenCV - the Viola-Jones Haar cascade and the HOG + linear-SVM people detector - are run against a labelled ground truth. You will compute IoU, build the precision/recall curve by sweeping the detector's own detector-specific operating parameter (not a calibrated probability), and report the operating point you would actually deploy.

What you will produce

out/detections.json with every predicted box, out/evaluation.csv with TP/FP/FN, precision, recall and F1 at three IoU thresholds, and a stated operating point with the parameter values that produce it.

Tools: Python 3 | opencv-python | NumPy - both detectors ship inside the base wheel, so no model weights are downloaded and no paid service is used

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: python3, opencv-python, numpy.

API currency. Everything taught here runs on the base opencv-python wheel. Two things that appear in older tutorials are *not* in that wheel and are deliberately avoided: cv2.TrackerCSRT_create / cv2.TrackerKCF_create and the cv2.quality SSIM module, both of which live in opencv-contrib-python. Where SSIM is needed the course ships its own NumPy implementation.

Workflow

1. Load the frames and the ground-truth JSON
2. Run the Haar cascade and the HOG detector

3. Compute IoU and match predictions to truth
4. Sweep the detector parameter for a PR curve
5. State the deployed operating point

Files in this folder

Path	What it is
reference/frames/frame_00.png .. frame_11.png	Twelve 640x480 synthetic scene frames containing schematic face-like and person-like targets at known pixel positions (deterministically rendered with OpenCV - SIMULATED classroom assets, not photographs). Because the ground truth is generated with the frames, every box is exact.
reference/positive-control/retail-adults-reference.png	AI-generated realistic retail scene with two fictional adults; a separate face-detector positive-control illustration, not a photograph or representative validation set.
data/positive-control-truth.json	Approximate independent manual visual boxes for two faces and two people in the realistic synthetic reference; review annotation boundaries before performance claims.
data/ground-truth.json	Exact bounding boxes for every target in every frame, with a class label
data/eval-config.json	The IoU thresholds to report, the parameter sweep ranges, and the operating point selection rule
detect_eval.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom asset rendered deterministically with OpenCV, an AI-generated reference copied unchanged with a PROVENANCE.md beside it, or a real prerecorded sequence with its source and SHA-256 recorded. Nothing in this folder may be relabelled: a rendered animation is never presented as model output, and a simulated stand-in is never presented as a photograph.

Steps

1. Use a prepared Python/OpenCV/NumPy environment offline. Work in this lab folder; create out/. For fragments use one Python notebook/REPL with import cv2; img=cv2.imread('reference/frames/frame_00.png'); assert img is not None; gray=cv2.cvtColor(img,cv2.COLOR_BGR2GRAY). Model weights ship with OpenCV; schematic targets may yield no detections. Zero denominators use a reported 0.0 convention; no qualifying point means DO NOT DEPLOY.

2. Confirm the two detectors are available without any download. cv2.data.haarcascades points inside the installed package and holds the trained cascade XML files; cv2.HOGDescriptor_getDefaultPeopleDetector() returns the trained SVM coefficients.

```
   python3 -c "import cv2, os;p=cv2.data.haarcascades+'haarcascade_frontalface_default.xml';
print(os.path.exists(p));h=cv2.HOGDescriptor();h.setSVMDetector(cv2.
HOGDescriptor_getDefaultPeopleDetector());print('HOG detector loaded')"
```

3. Open data/ground-truth.json. Each entry is a frame name mapped to a list of boxes in [x, y, w, h] pixel form with a class label. Count the total number of ground-truth boxes per class and record it - recall is measured against that denominator.

```
   python3 -c "import json,collections;g=json.load(open('data/ground-truth.json'));
c=collections.Counter(b['class'] for v in g['frames'].values() for b in v);print(dict(c))"
```

4. Explain the Viola-Jones cascade in your report before running it, in four parts: Haar-like rectangle features (differences of summed rectangle intensities), the integral image (which makes any rectangle sum a four-lookup operation), AdaBoost (which selects the small subset of features that actually discriminate), and the cascade of stages (which rejects most background windows in the first few stages).

5. Run the cascade with default parameters and record the number of detections per frame. scaleFactor controls the image-pyramid step and minNeighbors controls how many overlapping hits are required before a detection is emitted.

```
    cas = cv2.CascadeClassifier(cv2.data.haarcascades +
                                'haarcascade_frontalface_default.xml')
    boxes = cas.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
```

6. Now explain HOG + SVM: the image is divided into cells, a histogram of oriented gradients is accumulated per cell, cells are block-normalised for illumination invariance, and the concatenated descriptor is classified by a linear SVM slid across the image at multiple scales. It is a different family from the cascade - gradient statistics rather than intensity differences.

7. Run the HOG people detector. Note that detectMultiScale on HOGDescriptor returns boxes AND weights; the weights are the SVM decision values and are what you sweep to build a PR curve.

```
    hog = cv2.HOGDescriptor()
    hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
    boxes, weights = hog.detectMultiScale(img, winStride=(8, 8), padding=(8, 8), scale=1.05)
```

8. Compare raw and adjusted HOG boxes. The supplied evaluator uses a dataset-specific 15% width and5% height shrink. It is a post-processing choice, not a universal correction: report it beside metrics, evaluate the raw boxes as a baseline, and do not tune it against a held-out test set. Some boxes may improve and others worsen.

```
    x, y, bw, bh = box
    pw, ph = int(0.15 * bw), int(0.05 * bh)
    corrected = [x + pw, y + ph, bw - 2 * pw, bh - 2 * ph]
```

9. Implement IoU from the definition rather than importing it. For two boxes given as [x, y, w, h], the intersection width is max(0, min(x1+w1, x2+w2) - max(x1, x2)) and similarly for height; IoU = intersection area / (area1 + area2 - intersection area). Test it on two boxes you can verify by hand.

```
    def iou(a, b):
        ax2, ay2 = a[0] + a[2], a[1] + a[3]
        bx2, by2 = b[0] + b[2], b[1] + b[3]
        iw = max(0, min(ax2, bx2) - max(a[0], b[0]))
        ih = max(0, min(ay2, by2) - max(a[1], b[1]))
        inter = iw * ih
        union = a[2] * a[3] + b[2] * b[3] - inter
        return inter / union if union else 0.0
```

10. Hand-verify: box A = [0,0,100,100] and box B = [50,0,100,100] overlap on half their width, so intersection = 50*100 = 5000, union = 10000 + 10000 - 5000 = 15000, and IoU = 0.333. Confirm your function returns that.

11. Implement greedy matching. Sort predictions by score, and for each prediction take the highest-IoU unmatched ground-truth box of the SAME CLASS at or above the IoU threshold. Matched = true positive; unmatched prediction = false positive; unmatched ground truth = false negative. Each ground-truth box may be matched at most once - that rule is what stops a detector scoring well by emitting the same box ten times.

12. Compute precision = TP / (TP + FP) and recall = TP / (TP + FN), then F1 = 2PR / (P + R). Write the three formulas in your report; the assessor asks for them, not just the numbers.

13. Run the evaluator at three IoU thresholds: 0.3 (loose localisation), 0.5 (an example default) and 0.7 (tight). Record whether precision and recall change as the threshold tightens.

```
   python3 detect_eval.py --frames reference/frames --truth data/ground-truth.json --config
data/eval-config.json --out out
```

14. Run the separate realistic synthetic positive control with the command below. Keep its metrics separate from the twelve schematic frames. It contains one image, two manually annotated faces and two people; annotations are approximate, not detector-derived. The face detector may find these faces; HOG person detections are not guaranteed. Inspect overlays and report actual class-specific TP/FP/FN. This checks pipeline operation, not deployment quality or population accuracy.

```
   python3 detect_eval.py --frames reference/positive-control --truth data/positive-control-
truth.json --config data/eval-config.json --out out/positive-control
```

15. Sweep minNeighbors from 1 to 8 for the cascade and record precision and recall at each value. Lower settings may emit more detections; higher settings may suppress them. Neither precision improvement nor strict monotonicity is guaranteed. Plot or tabulate the pairs.

16. Sweep the HOG SVM weight threshold across the range in data/eval-config.json and record the same pairs. This gives you a score-threshold precision/recall table on a fixed HOG candidate pool rather than a single point.

17. Select and state your operating point using the rule in data/eval-config.json. The rule is not 'highest F1' by default - read it. For a review-queue application recall matters more than precision, because a missed target never reaches a human at all.

18. Write the honest limitations paragraph. State that these are synthetic schematic targets, that the numbers therefore describe the detectors on THIS distribution only, and that published detector accuracies are dataset-specific. Note as a reference point that published deepfake-detection backbones report validation accuracies in the mid-to-high 70s on hard, shifted data while AI-generated-image classifiers report 99%+ on curated benchmarks - the same metric, very different problems.

19. Save out/detections.json, out/evaluation.csv, your PR sweep table and the stated operating point, then complete the evidence checklist.

Verify

out/evaluation.csv reports TP, FP, FN, precision, recall and F1 for both detectors at IoU 0.3, 0.5 and 0.7; your hand-computed IoU of 0.333 matches your function; and you have stated one operating point with the exact parameter values that produce it and the selection rule you applied.

Expected outputs

- out/detections.json - every predicted box with its frame, class and score.
- out/evaluation.csv - one row per (detector, IoU threshold) with TP/FP/FN/P/R/F1.
- Observed metrics at all IoU thresholds, including legitimate unchanged values.
- A minNeighbors operating-parameter sweep; report actual values without forcing a trend.
- A HOG score-threshold sweep; precision need not be monotone.
- out/overlay_frame_00.png - a frame with ground truth in one colour and predictions in another.

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

[ ] The four-part explanation of the Viola-Jones cascade in your own words.

[ ] Your IoU function and the hand-verified 0.333 result.

[ ] The precision, recall and F1 formulas written out.

[ ] out/evaluation.csv at all three IoU thresholds.

[ ] The parameter sweep table for at least one detector.

[ ] Your stated operating point, the parameter values, and the selection rule you applied.

[ ] The limitations paragraph naming the synthetic distribution. Include out/positive-control/evaluation.csv and overlay, with separate sample size (one image/two faces/two people) and manual-annotation uncertainty.

Troubleshooting

Symptom	What to do
The cascade returns zero detections on every frame	Zero detections on schematic targets are a valid distribution-shift finding. Do not invent detections or select a deployable point if no operating point qualifies. As a documented exploratory comparison, lower minNeighbors to 2-3 and minSize to (20,20), and confirm you passed a GRAYSCALE image - detectMultiScale on input conventions should be verified for the chosen API.
cv2.data.haarcascades raises AttributeError	You are on a very old wheel. pip install --upgrade opencv-python. Do not download cascade XML files from the internet; they ship in the package.
HOG returns an empty weights array	Some builds return an empty tuple when no detection passes. Guard with if len(boxes) == 0: continue before indexing weights.
Recall is above 1.0	You matched one ground-truth box to more than one prediction. Enforce the match-at-most-once rule with a matched set keyed by ground-truth index.
Precision and recall are identical at every IoU threshold	This is possible (for example zero detections or all boxes above 0.7). Check that the threshold is passed into the matcher and not hard-coded.
The two detectors are compared as if they do the same job	They do not: the cascade here targets face-like regions and HOG targets person-like regions. Evaluate each against its own class in the ground truth, and say so.

References used in this lab

- [S16] Mewada et al. (2026), as above - *Ch.4 pp.47-54 (Sibi, Pantola and Gupta, 'Detecting AI-generated Images in the Social Media Era: A Deep Learning Approach with GenReal Dataset')*.

Detection framed as a supervised binary classification problem with Grad-CAM explainability; reported test accuracies on the GenReal dataset - EfficientFormer 99.6%, FastViT and CoAt-Lite 99.1%, CoAtNet and ConvNeXt 98.7% - and on CIFAKE - CoAt-Lite Mini 98.04%, ConvNeXt 97.95%, EfficientFormer 97.75%; macro precision, recall and F1 reported alongside accuracy.

- [S17] Mewada et al. (2026), as above - *Ch.7 pp.94-98 (Dubey, Pinheiro, Singh, Ansari and Kumar, 'Advanced Foundations and Future Trends in Generative AI for Visual Media')*.

Table 7.2 scaling strategies and their targets (trajectory distillation -> fewer sampling steps at similar FID/FVD; quantisation 8/4-bit -> reduced latency and memory; low-rank adapters -> small trainable-parameter fraction; latent diffusion -> lower bandwidth cost at high resolution; retrieval conditioning -> higher faithfulness without more

parameters). Evaluation protocol: FID as a 2-Wasserstein approximation in feature space (encoder-dependent, sample-size sensitive), KID as an unbiased MMD^2, LPIPS as a learned perceptual distance, precision/recall for mode coverage, CLIP-based text-image faithfulness, masked-region consistency and identity preservation for edits, FVD plus optical-flow agreement for video, and watermark detectability under common edits (Table 7.3 axes and pitfalls).

- [S19] Mewada et al. (2026), as above - *Ch.11 pp.147-160 (Dewang, Mewada, Omkar, Jaiswal and Singh, 'Hybrid Neural Networks for Robust Deepfake Detection')*.

Reported validation accuracies for deepfake detection backbones - Xception 77.5%, DenseNet121 and ACNN-RAN 75.5%, VGG19 73.5% - with precision, recall, F1 and AUC reported together, illustrating that headline accuracy alone hides class-wise behaviour on a hard, shifted distribution.

- [S23] OpenCV 4.13 Python package, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92)*.

API-currency verification for every command taught: cv2.data.haarcascades ships the frontal-face and eye cascades so no weights download is required; cv2.HOGDescriptor_getDefaultPeopleDetector() is built in; SIFT_create, ORB_create, FastFeatureDetector_create, cornerHarris, goodFeaturesToTrack, matchTemplate, createBackgroundSubtractorMOG2/KNN, meanShift, CamShift, calcOpticalFlowFarneback and PSNR are all present. IMPORTANT CORRECTION carried into the labs: cv2.TrackerCSRT_create and cv2.TrackerKCF_create are NOT in the base opencv-python wheel (they require opencv-contrib-python), and the cv2.quality SSIM module is also contrib-only - so the labs use cv2.TrackerMIL_create and a NumPy SSIM implementation to stay dependency-light and runnable offline.
